Java practice programs list

Data Types, Variables & Operators (20 Practice Problems)

Variables & Data Types

1. Write a program to store and display your personal information (name, age, height, weight, CGPA, gender) using appropriate data types.
2. Declare one variable of every primitive data type and print their values.
3. Find the size (in bytes) and range of every primitive data type using wrapper class constants.
4. Swap two numbers using a third variable and then without using a third variable.
5. Demonstrate the difference between local, instance, and static variables in one program.

Constants & Literals

6. Write a program using `final` to calculate the area and circumference of a circle.
7. Create a program that demonstrates integer, floating-point, character, string, boolean, hexadecimal, binary, and octal literals.
8. Write a program to show what happens when you try to modify a `final` variable.

Type Casting

9. Convert all primitive numeric data types into each other using explicit casting and display the results.
10. Demonstrate implicit type conversion with different arithmetic expressions.
11. Show the data loss that occurs when converting `double` to `int`, `long` to `short`, and `int` to `byte`.

Operators

12. Build a simple calculator using arithmetic operators (`+`, `-`, `*`, `/`, `%`).
13. Demonstrate pre-increment and post-increment operators using different expressions and print every intermediate value.
14. Compare two numbers using relational operators and print the result of every comparison.

15. Write a program to check whether a person is eligible for voting and a driving license using logical operators.
16. Perform all bitwise operations (`&`, `|`, `^`, `~`) on two integers and display both decimal and binary results.
17. Demonstrate left shift (`<<`) and right shift (`>>`) operators on different numbers and explain the output through comments.
18. Use the ternary operator to find the largest of two numbers without using `if`.
19. Create a program that demonstrates operator precedence by evaluating a complex arithmetic expression step by step.
20. Build a **Student Result Calculator** that:

- Accepts marks of 5 subjects.
- Calculates total and percentage.
- Uses constants where appropriate.
- Uses arithmetic, relational, logical, and ternary operators.
- Displays Pass/Fail based on percentage.

Control Flow Statements – 20 Practice Problems

Part A – Conditional Statements (`if`, `if-else`, `switch`)

1. Positive, Negative or Zero ⋆

Write a program to determine whether a number is:

- Positive
- Negative
- Zero

2. Even or Odd ⋆

Determine whether a given integer is even or odd.

3. Largest of Three Numbers ⋆⋆

Find the largest among three numbers using nested `if`.

4. Leap Year Checker ⋆⋆

Determine whether a given year is a leap year.

5. Student Grade Calculator ⋆⋆

Input marks and display:

- A
- B
- C
- D
- F

using `if-else`.

6. Calculator using `switch` ⋆⋆

Accept:

- Two numbers
- Operator (`+`, `-`, `*`, `/`, `%`)

Perform the selected operation using `switch`.

7. Month Information ⋆⋆⋆

Input a month number (1–12).

Display:

- Month Name
- Number of Days

Use `switch`.

Part B – Looping Statements (`for`, `while`, `do-while`)

8. Print Numbers ⋆

Print numbers from **1 to N**.

9. Sum of First N Numbers ⋆

Calculate the sum of the first **N** natural numbers.

10. Multiplication Table ⋆⋆

Print the multiplication table of a given number up to 20.

11. Factorial Calculator ⋆⋆

Calculate the factorial of a number using:

- `for`
- `while`

12. Reverse a Number ⋆⋆⋆

Reverse a given integer.

Example:

```
Input:
12345

Output:
54321
```

13. Palindrome Number ⋆⋆⋆

Check whether a number is a palindrome.

14. Prime Number Checker ⋆⋆⋆

Determine whether a given number is prime.

15. Prime Numbers in a Range ⋆⋆⋆⋆

Print all prime numbers between two given numbers.

Part C – Jump Statements (`break`, `continue`, `return`)

16. Number Guessing Game ⋆⋆⋆

The user keeps entering numbers until the correct number is guessed.

Use:

- Loop
- `break`

17. Skip Multiples of 3 ⋆⋆⋆

Print numbers from 1 to 100.

Skip every multiple of 3 using `continue`.

18. Menu-Driven Calculator ★★★★

Create a menu:

```
1. Add
2. Subtract
3. Multiply
4. Divide
5. Exit
```

Requirements:

- Keep displaying the menu until the user selects **Exit**.
- Use `switch`, loops, and `break`.

19. Login System ★★★★

Requirements:

- Username: `admin`
- Password: `java123`
- Maximum 3 attempts
- Display **Account Locked** after 3 failed attempts.
- Exit immediately after successful login.

Concepts:

- `if`
- Loops
- `break`
- `return`

Part D – Chapter Assignment ★★★★★

20. ATM Console System

Build a menu-driven ATM application.

Menu:

```
1. Check Balance
2. Deposit
3. Withdraw
4. Change PIN
```

```
5. Mini Statement
6. Exit
```

Requirements:

- Initial balance: ₹10,000
- Initial PIN: 1234
- Verify PIN before every transaction.
- Deposit should increase balance.
- Withdrawal should fail if balance is insufficient.
- Allow PIN change after verifying the old PIN.
- Display the last 5 transactions (you can implement this using simple variables for now; later you'll improve it using arrays).
- Keep showing the menu until the user chooses **Exit**.

Arrays – 20 Practice Problems

Part A – Single Dimensional Arrays

1. Array Input & Output ⋆

Create an integer array of size **N**, accept elements from the user, and display them.

2. Sum and Average ⋆

Find the sum and average of all elements in an array.

3. Maximum and Minimum ⋆

Find the largest and smallest element in an array.

4. Count Even and Odd Numbers ⋆⋆

Count how many even and odd numbers are present in the array.

5. Search an Element ⋆⋆

Search for a given element using linear search and display its index if found.

6. Reverse an Array ⋆⋆

Reverse the elements of an array without using another array.

7. Second Largest Element ⋆⋆⋆

Find the second largest element without sorting the array.

8. Remove Duplicate Elements ⋆⋆⋆

Print only the unique elements from an array.

9. Frequency of Each Element ⋆⋆⋆

Count how many times each element appears in the array.

10. Left and Right Rotation ⋆⋆⋆

Rotate the array:

- Left by one position
- Right by one position

Part B – Multi-Dimensional Arrays

11. Matrix Input & Output ⋆⋆

Accept a **3 × 3** matrix and display it in matrix form.

12. Matrix Addition ⋆⋆⋆

Add two matrices of the same size.

13. Matrix Multiplication ⋆⋆⋆⋆

Multiply two matrices.

14. Transpose of a Matrix ⋆⋆⋆

Find the transpose of a matrix.

15. Sum of Rows and Columns ⋆⋆⋆

Display:

- Sum of each row
- Sum of each column

Part C – Array Class & Methods

16. Sort an Array ⋆⋆⋆

Sort an array using `Arrays.sort()` and display it in ascending order.

17. Compare Two Arrays ⋆⋆⋆

Check whether two arrays are equal using the `Arrays` class.

18. Copy an Array ⋆⋆⋆⋆

Create copies of an array using:

- `Arrays.copyOf()`
- `Arrays.copyOfRange()`

Display the results.

Part D – For-each Loop

19. Employee Salary Analyzer ⋆⋆⋆

Store employee salaries in an array.

Using a **for-each loop**:

- Display all salaries
- Calculate total salary
- Find average salary
- Count employees earning above the average

Part E – Chapter Assignment ⋆⋆⋆⋆⋆

20. Student Record Management System

Create a console-based application to manage student records.

Requirements:

- Store details of **N students**:
    - Roll Number
    - Name

    - Marks in 5 subjects
- Calculate:
    - Total
    - Percentage
    - Grade
- Find:
    - Highest scorer
    - Lowest scorer
    - Class average
- Search a student by roll number.
- Sort students based on total marks.
- Display all student records neatly.

Concepts Covered:

- Single-dimensional arrays
- Two-dimensional arrays (for marks)
- `Arrays.sort()` (or manual sorting if preferred)
- For-each loop
- Searching
- Traversing
- Array manipulation

Object-Oriented Programming (OOP) – 20 Practice Problems

Part A – Classes & Objects

1. Student Class ⭑

Create a `Student` class with:

- Roll Number
- Name
- Age
- CGPA

Create 3 objects and display their details.

2. Employee Class ⭑

Create an `Employee` class.

Store:

- Employee ID
- Name
- Department
- Salary

Create multiple employee objects and display them.

3. Bank Account ⋆⋆

Create a `BankAccount` class.

Features:

- Deposit
- Withdraw
- Check Balance

Create multiple accounts.

Part B – Constructors

4. Constructor Demonstration ⋆⋆

Create a class with:

- Default constructor
- Parameterized constructor

Show the difference between them.

5. Constructor Overloading ⋆⋆

Create a `Book` class with multiple constructors.

Examples:

- Empty constructor
- Title only
- Title + Author
- Title + Author + Price

6. Constructor Chaining ⋆⋆⋆

Use `this()` to call one constructor from another.

Display the order of execution.

Part C – this Keyword

7. Resolve Variable Shadowing ⋆⋆

Create a class where constructor parameters have the same names as instance variables.

Use `this` to initialize them correctly.

8. Use `this.method()` ⋆⋆⋆

Create a class where one method calls another using `this`.

Part D – Inheritance

9. Single Inheritance ⋆⋆

Create:

- `Person`
- `Student`

Student should inherit from Person.

Display inherited properties.

10. Multilevel Inheritance ⋆⋆⋆

Create:

```
Person
↓
Employee
↓
Manager
```

Display inherited members.

11. Hierarchical Inheritance ⋆⋆⋆

Create:

```
Vehicle
```

↓

```
Car
Bike
Bus
```

Each class should have its own method.

Part E – Method Overloading & Overriding

12. Method Overloading ★★★

Create a `Calculator` class.

Overload methods for:

- Two integers
- Three integers
- Two doubles

13. Method Overriding ★★★

Create:

```
Animal
```

↓

```
Dog
```

Override `makeSound()`.

Call methods using parent and child references.

Part F – super Keyword

14. Parent Constructor ★★★

Use `super()` to invoke the parent constructor.

Print the execution order.

15. Parent Method ★★★

Override a method.

Use `super.method()` inside the child class.

Part G – final Keyword

16. final Variable, Method & Class ★★★

Create examples showing:

- final variable
- final method
- final class

Observe what is allowed and what causes compilation errors.

Part H – Static Members

17. Static Members ★★★★

Create a `Student` class with:

- Static college name
- Instance student details

Show that changing the static variable affects all objects.

Also include:

- Static method
- Static block

Part I – Access Modifiers

18. Access Modifier Demonstration ★★★★

Create classes to demonstrate:

- public
- private
- protected
- default

Access members from:

- Same class
- Same package
- Different class

Part J – Mixed OOP Challenge

19. Library Management System ★★★★

Create classes:

- Book
- Member
- Library

Requirements:

- Constructors
- Static member for total books
- Inheritance (PremiumMember)
- Method overloading
- Method overriding
- Access modifiers
- `this`
- `super`
- `final` where appropriate

Chapter Assignment ★★★★★

20. College Management System

Build a console-based application.

Classes

- Person
- Student
- Faculty
- Course
- Department

Requirements

- Create objects using constructors.
- Use constructor overloading.
- Use `this` for initialization.
- Use inheritance for Student and Faculty.
- Override methods like `displayDetails()`.
- Overload methods like `searchStudent()`.
- Use `super` to access parent constructors and methods.
- Use `final` for constants such as maximum marks.
- Use static members to count total students and faculty.
- Apply proper access modifiers.
- Display complete college information through object interactions.

Encapsulation, Abstraction & Polymorphism – 20 Practice Problems

Part A – Encapsulation

1. Student Encapsulation ⋆

Create a `Student` class with:

- private fields
- public getters
- public setters

Create objects and display student details.

2. Bank Account ⋆⋆

Create a `BankAccount` class.

Requirements:

- Balance should be private.
- Deposit method.
- Withdraw method.
- Getter for balance.
- Prevent negative balance.

3. Employee Salary System ⋆⋆

Create an `Employee` class.

Private fields:

- id
- name
- salary

Do not allow salary to become negative.

4. Password Manager ⋆⋆⋆

Create a class that stores a password privately.

Requirements:

- Change password only after verifying the old password.
- Password cannot be empty.

5. Mobile Phone Class ⋆⋆⋆

Store:

- Brand
- Model
- IMEI Number
- Price

Make fields private.

Allow modification only through setter methods with validation.

Part B – Abstraction (Abstract Classes)

6. Shape Calculator ⋆⋆

Create an abstract class `Shape`.

Abstract method:

- area()

Create:

- Circle

- Rectangle
- Triangle

7. Vehicle System ⋆⋆⋆

Create an abstract class `Vehicle`.

Abstract methods:

- start()
- stop()

Implement:

- Car
- Bike
- Bus

8. Employee Payroll ⋆⋆⋆

Create an abstract class `Employee`.

Abstract method:

- calculateSalary()

Implement:

- FullTimeEmployee
- PartTimeEmployee

9. Online Payment System ⋆⋆⋆

Create an abstract class `Payment`.

Implement:

- UPI
- Credit Card
- Net Banking

Each payment method should process payment differently.

10. Hospital Management ⋆⋆⋆⋆

Create an abstract class `Person`.

Implement:

- Doctor
- Nurse
- Patient

Each should override `displayRole()`.

Part C – Interfaces

11. Remote Control ⭑⭑

Create an interface `Remote`.

Methods:

- powerOn()
- powerOff()

Implement:

- TV
- AC

12. Payment Gateway ⭑⭑⭑

Create interface `PaymentGateway`.

Implement:

- GooglePay
- PhonePe
- Paytm

Each processes payment differently.

13. Animal Sounds ⭑⭑⭑

Create interface `Animal`.

Method:

- makeSound()

Implement:

- Dog
- Cat
- Cow

14. Smart Device ⋆⋆⋆⋆

Create two interfaces:

- Camera
- MusicPlayer

Create class `SmartPhone` implementing both interfaces.

15. University Portal ⋆⋆⋆⋆

Create interfaces:

- Attendance
- Examination

Student class should implement both.

Part D – Polymorphism

16. Compile-Time Polymorphism ⋆⋆⋆

Create a `Calculator`.

Overload methods for:

- int
- double
- float

17. Runtime Polymorphism ⋆⋆⋆

Create:

```
Animal

↓

Dog

Cat

Cow
```

Override `makeSound()`.

Call methods using parent reference.

18. Dynamic Method Dispatch ★★★★

Create:

```
Employee

↓

Manager

Developer

Tester
```

Store them using an `Employee` reference.

Call overridden methods.

Observe runtime polymorphism.

19. Shopping Cart ★★★★

Create interface:

```
Discount
```

Implement:

- StudentDiscount
- FestivalDiscount
- PremiumMemberDiscount

Use runtime polymorphism to calculate the final bill.

Chapter Assignment ★★★★★

20. Online Banking System

Build a console application.

Requirements

Create:

Abstract Class

`Account`

Methods:

- deposit()
- withdraw()
- displayBalance()

Child Classes

- SavingsAccount
- CurrentAccount

Interface

`LoanFacility`

Methods:

- applyLoan()
- calculateEMI()

Implement it where required.

Encapsulation

Keep:

- accountNumber
- balance
- customerName

private.

Use getters and setters.

Polymorphism

Store all accounts using the parent reference.

Call overridden methods.

Features

- Deposit
- Withdraw
- Balance Inquiry
- Loan Application
- Account Details

String Handling – 20 Practice Problems

Part A – String Basics

1. Create and Display Strings ⋆

Create strings using:

- String literal
- `new String()`

Display both.

2. String Length ⋆

Accept a string and print its length.

3. Character Frequency ⋆⋆

Count:

- Alphabets
- Digits
- Special characters
- Spaces

4. Reverse a String ⋆⋆

Reverse a string without using built-in reverse methods.

5. Palindrome String ⋆⋆

Check whether a string is a palindrome.

Part B – String Methods

6. Count Vowels and Consonants ⋆⋆

Count vowels and consonants in a string.

7. Word Counter ⋆⋆⋆

Count the number of words in a sentence.

8. String Method Explorer ⋆⋆⋆

Using one input string, demonstrate:

- `length()`
- `charAt()`
- `substring()`
- `indexOf()`
- `lastIndexOf()`
- `contains()`
- `startsWith()`
- `endsWith()`

9. Replace Characters ⋆⋆⋆

Replace:

- Spaces with _
- One word with another
- One character with another

10. String Formatter ⋆⋆⋆

Input a sentence and display:

- Uppercase
- Lowercase
- Trimmed string
- First character capitalized

Part C – String Comparison

11. Compare Two Strings ★★

Compare two strings using:

- `==`
- `equals()`
- `equalsIgnoreCase()`

Observe the difference.

12. Dictionary Order ★★★

Accept two strings and compare them using `compareTo()`.

Display which comes first alphabetically.

13. Username & Password Validation ★★★

Create a login system.

Compare username and password correctly.

Do **not** use `==`.

Part D – StringBuilder & StringBuffer

14. StringBuilder Operations ★★★

Demonstrate:

- append()
- insert()
- delete()
- replace()
- reverse()

15. StringBuffer Operations ★★★

Perform the same operations using `StringBuffer`.

16. Build a Paragraph ★★★★

Use `StringBuilder` to build a paragraph line by line instead of using `+`.

Display the final paragraph.

17. Menu-Driven Text Editor ★★★★

Using `StringBuilder`, implement:

- Append text
- Insert text
- Delete text
- Replace text
- Reverse text
- Exit

Part E – String Immutability

18. Demonstrate Immutability ★★★

Create a string.

Modify it using:

- `concat()`
- `replace()`

Print references before and after modification.

Observe why Strings are immutable.

19. String vs StringBuilder vs StringBuffer ★★★★

Write one program demonstrating:

- Modification
- Performance
- Mutability

Print observations as comments.

Chapter Assignment ★★★★★

20. Student Information Parser

Create a console application.

Input format:

```
101,Rahul,CSE,89,91,85,90,88
```

Requirements:

- Split the string.
- Store data.
- Calculate total.
- Calculate percentage.
- Assign grade.
- Display formatted student details.
- Validate input.
- Compare department names correctly.
- Use:
    - String methods
    - StringBuilder
    - String comparison
    - String immutability concepts

Exception Handling – 20 Practice Problems

Part A – Types of Exceptions

1. ArithmeticException ⭑

Create a program that divides two numbers.

Handle division by zero.

---

2. ArrayIndexOutOfBoundsException ⭑

Access an invalid array index and handle the exception.

---

3. NullPointerException ⭑⭑

Create a null reference and safely handle the exception.

---

4. NumberFormatException ⭑⭑

Accept a number as a string.

Handle invalid numeric input.

---

5. Multiple Exceptions ⋆⋆

Write one program that can produce:

- ArithmeticException
- ArrayIndexOutOfBoundsException
- NullPointerException

Handle each separately.

Part B – try, catch & finally

6. Calculator with Exception Handling ⋆⋆

Create a calculator.

Prevent program termination on invalid operations.

---

7. File Closing Simulation ⋆⋆⋆

Simulate opening and closing a resource.

Use the `finally` block to ensure cleanup.

---

8. Nested try-catch ⋆⋆⋆

Create nested `try-catch` blocks.

Generate different exceptions in inner and outer blocks.

---

9. Multiple Catch Blocks ⋆⋆⋆

Write one program using multiple `catch` blocks for different exception types.

---

10. ATM Withdrawal ⋆⋆⋆

Create an ATM program.

Handle:

- Invalid amount
- Division by zero (if any calculation exists)
- Invalid menu choice

Always display a closing message using `finally`.

Part C – throw

11. Age Validation ⋆⋆

If age is less than 18,

Throw an exception with an appropriate message.

---

12. Marks Validation ⋆⋆⋆

Accept marks.

If marks are:

- Less than 0
- Greater than 100

Throw an exception.

---

13. Bank Withdrawal ⋆⋆⋆

Throw an exception when withdrawal amount exceeds the account balance.

Part D – throws

14. Student Record Reader ⋆⋆⋆

Create a method that declares `throws Exception`.

Call it from `main()` and handle the exception there.

---

15. File Processing Simulation ⋆⋆⋆

Create multiple methods.

Propagate an exception using `throws` through the method chain.

Part E – Custom Exceptions

16. InvalidAgeException ⋆⋆⋆⋆

Create your own exception.

Throw it when age is below the required limit.

---

17. InsufficientBalanceException ⋆⋆⋆⋆

Create a custom exception for banking transactions.

---

18. InvalidPasswordException ⋆⋆⋆⋆

Create a custom exception.

Password rules:

- Minimum 8 characters
- At least one digit
- At least one uppercase letter

Throw the exception if validation fails.

19. Online Examination System ★★★★

Create custom exceptions for:

- Invalid Roll Number
- Invalid Marks
- Student Not Found

Handle all exceptions gracefully.

Chapter Assignment ★★★★★

20. Railway Reservation System

Build a console application.

Features

- Book Ticket
- Cancel Ticket
- View Ticket
- Exit

Exception Handling Requirements

Handle:

- Invalid menu option
- Invalid passenger age
- Invalid seat number
- Invalid ticket number
- Insufficient balance

Use:

- `try`
- `catch`

- `finally`
- `throw`
- `throws`
- At least **two custom exceptions**

The program should **never terminate unexpectedly** because of invalid user input.

Wrapper Classes – 20 Practice Problems

Part A – Wrapper Class Basics

1. Primitive to Wrapper ⋆

Create variables of all primitive data types and convert them into their corresponding wrapper objects.

---

2. Wrapper to Primitive ⋆

Create wrapper objects and convert them back into primitive data types.

---

3. Display Wrapper Information ⋆

Using wrapper classes, display:

- MIN_VALUE
- MAX_VALUE
- SIZE
- BYTES

for different primitive types.

---

4. String to Primitive ⋆⋆

Accept numeric values as strings and convert them into:

- int

- double
- float
- long

using wrapper classes.

---

5. Primitive Converter ⋆⋆

Accept different primitive values and display their wrapper object values.

Part B – Autoboxing

6. Automatic Boxing ⋆⋆

Create primitive variables and assign them directly to wrapper objects.

Display the converted values.

---

7. ArrayList with Integers ⋆⋆⋆

Store primitive integers in an `ArrayList<Integer>`.

Observe autoboxing while adding elements.

---

8. Sum of Wrapper Objects ⋆⋆⋆

Store multiple integers in an `ArrayList<Integer>` and calculate the sum.

Observe automatic boxing during insertion.

---

9. Student Marks System ⋆⋆⋆

Store marks using `ArrayList<Integer>`.

Calculate:

- Total
- Average
- Highest
- Lowest

---

10. Wrapper Comparison ⋆⋆⋆

Compare wrapper objects using:

- `==`
- `equals()`

Observe the difference.

Part C – Unboxing

11. Automatic Unboxing ⋆⋆

Store wrapper objects.

Assign them directly to primitive variables.

Display the results.

---

12. Arithmetic with Wrapper Objects ⋆⋆⋆

Perform arithmetic operations directly on wrapper objects.

Observe automatic unboxing.

---

13. Average Calculator ⋆⋆⋆

Store wrapper objects inside an array or `ArrayList`.

Calculate the average using primitive arithmetic.

---

14. Temperature Analyzer ⋆⋆⋆

Store temperatures as `Double`.

Calculate:

- Highest
- Lowest
- Average

using automatic unboxing.

---

15. Boolean Wrapper ⋆⋆⋆

Create a simple login system using `Boolean` wrapper objects instead of primitive `boolean`.

Part D – Mixed Wrapper Class Problems

16. Number Analyzer ⋆⋆⋆⋆

Accept a string.

Determine whether it can be converted into:

- Integer
- Double
- Float
- Long

Handle invalid input gracefully.

---

17. Wrapper Utility Program ⋆⋆⋆⋆

Create a menu-driven application.

Menu:

```
1. String → Integer
2. Integer → String
3. Double → Integer
4. Character → String
5. Exit
```

Perform the selected conversion.

---

18. Wrapper Class Explorer ★★★★

Demonstrate the use of:

- `Integer`
- `Double`
- `Float`
- `Character`
- `Boolean`

Use methods such as:

- `parseXxx()`
- `valueOf()`
- `toString()`

---

19. Student Result Processor ★★★★

Accept all input as **String**.

Convert values using wrapper classes.

Calculate:

- Total
- Percentage
- Grade

Chapter Assignment ★★★★★

20. Banking Transaction Analyzer

Create a console application.

Requirements

- Accept all user input as **String**.
- Convert values using wrapper classes.
- Store balances using wrapper objects.
- Perform deposits and withdrawals.
- Calculate final balance.
- Compare account numbers.
- Display account statistics.

Concepts Covered

- Wrapper Classes
- Autoboxing
- Unboxing
- Primitive ↔ Object Conversion
- `parseXxx()`
- `valueOf()`
- `toString()`
- Wrapper object comparison

Java Input/Output (I/O) – 20 Practice Problems

Part A – File Handling (`File`, `FileReader`, `FileWriter`)

1. Create a File ⭐

Write a program to create a new text file. Display whether the file was created successfully.

---

2. File Information ⭐

Display:

- File Name
- Absolute Path

- File Size
- Can Read?
- Can Write?
- Is Hidden?

---

3. Write to a File ⋆⋆

Accept user input and write it into a text file using `FileWriter`.

---

4. Read a File ⋆⋆

Read the contents of a text file using `FileReader` and display them on the console.

---

5. Append Data ⋆⋆

Append new text to an existing file without deleting its previous contents.

Part B – Character Streams

6. Copy a Text File ⋆⋆⋆

Copy the contents of one text file into another using `FileReader` and `FileWriter`.

---

7. Count Characters, Words and Lines ⋆⋆⋆

Read a text file and display:

- Total Characters
- Total Words
- Total Lines

---

8. Search a Word ⋆⋆⋆

Search for a user-given word inside a text file and display how many times it appears.

---

9. Convert Case ⋆⋆⋆

Read a text file and create another file where all text is converted to uppercase.

---

10. Student Report Generator ⋆⋆⋆

Store student details in a text file and read them back in a formatted report.

Part C – Byte Streams

11. Copy an Image ⋆⋆⋆

Copy an image file using `FileInputStream` and `FileOutputStream`.

Verify that the copied image opens correctly.

---

12. File Size Calculator ⋆⋆⋆

Display the size of any selected file in:

- Bytes
- KB
- MB

---

13. Binary File Copier ⋆⋆⋆⋆

Copy any binary file (PDF, image, video, etc.) using byte streams.

Part D – Buffered Streams

14. Buffered Text Reader ★★★

Read a text file using `BufferedReader` and display it line by line.

---

15. Buffered Writer ★★★

Write multiple lines into a file using `BufferedWriter`.

---

16. Fast File Copy ★★★★

Copy a large text file using buffered streams and compare it with normal file copying.

Part E – Serialization & Deserialization

17. Serialize a Student Object ★★★★

Create a `Student` class.

Store:

- Roll Number
- Name
- Age
- CGPA

Serialize the object into a file.

---

18. Deserialize a Student Object ★★★★

Read the serialized student object and display all details.

---

19. Employee Database ★★★★

Create multiple `Employee` objects.

Serialize them into a file.

Deserialize them later and display the records.

Chapter Assignment ★★★★★

20. Student Record Management System

Build a console application.

Features

- Add Student
- View Students
- Search Student
- Update Student
- Delete Student
- Exit

Requirements

- Store records in files.
- Read records from files.
- Use `BufferedReader` and `BufferedWriter` where appropriate.
- Serialize and deserialize `Student` objects.
- Handle file-related exceptions properly.
- Display meaningful messages if files are missing or corrupted.

Concepts Covered

- `File`
- `FileReader`
- `FileWriter`
- `FileInputStream`
- `FileOutputStream`
- `BufferedReader`
- `BufferedWriter`
- Serialization
- Deserialization
- Basic file management

Collections Framework – 20 Practice Problems

Part A – List (`ArrayList`, `LinkedList`)

1. Student List ⭐

Store the names of 10 students in an `ArrayList`.

Display all names.

---

2. Employee Management ⭐⭐

Store employee objects in an `ArrayList`.

Display:

- All employees
- Total employees

---

3. Shopping Cart ⭐⭐

Create a shopping cart using `ArrayList`.

Features:

- Add item
- Remove item
- Update item
- View cart

---

4. LinkedList Playlist ⭐⭐⭐

Create a music playlist using `LinkedList`.

Features:

- Add song
- Remove song
- Display playlist
- Play next song

Part B – Set (`HashSet`, `LinkedHashSet`, `TreeSet`)

5. Unique Roll Numbers ★★

Store roll numbers using `HashSet`.

Prevent duplicate entries.

---

6. Website Visitor Log ★★★

Store unique visitor IDs using `HashSet`.

Display the total unique visitors.

---

7. Ordered Cities ★★★

Store city names using `LinkedHashSet`.

Maintain insertion order.

---

8. Leaderboard ★★★

Store player scores using `TreeSet`.

Display scores in ascending order.

Part C – Map (`HashMap`, `LinkedHashMap`, `TreeMap`)

9. Student Marks ★★★

Store:

- Roll Number → Marks

using `HashMap`.

Search marks by roll number.

---

10. Phone Book ★★★

Store:

- Name → Phone Number

using `LinkedHashMap`.

Display contacts in insertion order.

---

11. Dictionary ★★★★

Store:

- Word → Meaning

using `TreeMap`.

Display the dictionary alphabetically.

---

12. Product Inventory ★★★★

Store:

- Product ID → Product Object

Perform:

- Add
- Search
- Update
- Remove

Part D – Queue & Stack

13. Printer Queue ★★★

Implement a printer queue using `Queue`.

Features:

- Add document
- Print next document
- View pending documents

---

14. Hospital Token System ★★★

Use `Queue` to manage patient tokens.

---

15. Browser History ★★★★

Use `Stack` to implement:

- Visit page
- Back
- Display history

---

16. Undo Operation ★★★★

Implement an Undo feature for a simple text editor using `Stack`.

Part E – Iterators

17. Employee Iterator ★★★

Store employees in an `ArrayList`.

Traverse using:

- `Iterator`
- `ListIterator`

Compare both approaches.

---

18. Remove Even Numbers ★★★★

Store integers in an `ArrayList`.

Use `Iterator` to remove all even numbers safely.

Part F – Collections Utility Class

19. Collections Utility Explorer ★★★★

Create an `ArrayList<Integer>`.

Demonstrate:

- `sort()`
- `reverse()`
- `shuffle()`
- `max()`
- `min()`
- `frequency()`
- `binarySearch()`

Chapter Assignment ★★★★★

20. Library Management System

Build a console application.

Features

- Add Book
- Remove Book
- Search Book
- Issue Book

- Return Book
- Display All Books
- Exit

Requirements

Use different collections appropriately:

- **ArrayList** → Store all books
- **HashMap** → Book ID → Book
- **HashSet** → Issued Book IDs (avoid duplicates)
- **Queue** → Book reservation requests
- **Stack** → Recently viewed books
- **Iterator** → Traverse and remove books
- **Collections Utility Class** → Sort books by title

Concepts Covered

- ArrayList
- LinkedList
- HashSet
- LinkedHashSet
- TreeSet
- HashMap
- LinkedHashMap
- TreeMap
- Queue
- Stack
- Iterator
- ListIterator
- Collections Utility Class

Generics – 20 Practice Problems

Part A – Generic Classes

1. Generic Box ⋆

Create a generic class `Box<T>` that stores and displays a single value.

---

2. Generic Student Record ⋆⋆

Create a generic class to store:

- Integer Roll Number
- String Name
- Double CGPA

Create multiple objects with different data types.

---

3. Generic Pair ⋆⋆

Create a generic class `Pair<K, V>` to store key-value pairs.

Example:

- Roll No → Name
- Product ID → Price

---

4. Generic Stack ⋆⋆⋆

Create a generic stack class with methods:

- push()
- pop()
- peek()

---

5. Generic Queue ⋆⋆⋆

Create a generic queue class with methods:

- enqueue()
- dequeue()
- display()

Part B – Generic Methods

6. Generic Display Method ⋆⋆

Write a generic method to display elements of an array of any data type.

---

7. Generic Swap Method ⋆⋆⋆

Write a generic method to swap two values.

---

8. Generic Maximum Finder ⋆⋆⋆

Write a generic method that returns the largest of two values.

---

9. Generic Array Printer ⋆⋆⋆

Accept arrays of:

- Integer
- Double
- String

Display them using one generic method.

---

10. Generic Calculator ⋆⋆⋆

Create generic methods for:

- Display
- Compare
- Find maximum

Part C – Bounded Type Parameters

11. Number Calculator ⋆⋆⋆

Create a generic class that accepts only subclasses of `Number`.

Calculate the sum of two values.

---

12. Average Calculator ⋆⋆⋆

Write a generic method that calculates the average of numeric arrays.

Use bounded type parameters.

---

13. Student Marks Analyzer ⋆⋆⋆⋆

Accept only numeric marks.

Calculate:

- Total
- Average
- Highest

using bounded generics.

---

14. Generic Statistics ⋆⋆⋆⋆

Create a generic class that accepts only numeric data and displays:

- Minimum
- Maximum
- Average

Part D – Wildcards

15. Upper Bounded Wildcard (`? extends`) ⋆⋆⋆

Create a method that accepts a list of any subclass of `Number` and prints all elements.

---

16. Lower Bounded Wildcard (`? super`) ⋆⋆⋆⋆

Create a method that adds integers into a list using `? super Integer`.

---

17. Generic List Printer ★★★★

Write one method that can display:

- `ArrayList<Integer>`
- `ArrayList<Double>`
- `ArrayList<String>`

using wildcards.

---

18. Generic Library ★★★★

Create:

- `Book`
- `Magazine`

Store them in generic collections.

Write methods using wildcards to display all items.

---

19. Generic Employee Management ★★★★

Create a generic repository that stores:

- Employee
- Student
- Product

Write reusable methods using generics.

Chapter Assignment ★★★★★

20. Generic Inventory Management System

Build a console application.

Requirements

Create a generic class:

`Inventory<T>`

Support:

- Add Item
- Remove Item
- Search Item
- Display Items

Store different object types:

- Product
- Book
- Student
- Employee

Use:

- Generic Classes
- Generic Methods
- Bounded Type Parameters
- `? extends`
- `? super`

Demonstrate how the same inventory system works for different object types without rewriting code.

Multithreading & Concurrency – 20 Practice Problems

Part A – Thread Class & Runnable Interface

1. Your First Thread ⋆

Create a thread by extending the `Thread` class.

Print numbers from 1 to 10.

---

2. Runnable Interface ⭑⭑

Create a thread by implementing the `Runnable` interface.

Print your name 10 times.

---

3. Multiple Threads ⭑⭑

Create two threads:

- Thread A → Prints even numbers.
- Thread B → Prints odd numbers.

Run them simultaneously.

---

4. Thread Information ⭑⭑

Create a thread and display:

- Thread Name
- Thread ID
- Priority
- State

---

5. Number Printer ⭑⭑⭑

Create three threads.

Each thread prints a different range:

- 1–10
- 11–20
- 21–30

Part B – Thread Life Cycle & Thread Methods

6. Using `sleep()` ⭐⭐

Create a thread that prints numbers with a 1-second delay.

---

7. Using `join()` ⭐⭐⭐

Create two threads.

Ensure the second thread starts only after the first finishes.

---

8. Thread Priority ⭐⭐⭐

Create three threads with different priorities.

Observe their execution order.

---

9. Countdown Timer ⭐⭐⭐

Create a countdown timer from 10 to 1 using `sleep()`.

---

10. Sequential Task Execution ⭐⭐⭐

Create three threads representing:

- Download
- Installation
- Launch

Execute them using `join()` in the correct order.

Part C – Thread Synchronization

11. Synchronized Counter ⋆⋆⋆

Create multiple threads incrementing the same counter.

Synchronize the counter method.

---

12. Bank Account ⋆⋆⋆

Two threads withdraw money from the same account.

Use synchronization to prevent incorrect balance updates.

---

13. Ticket Booking System ⋆⋆⋆⋆

Simulate multiple users booking seats simultaneously.

Prevent double booking using synchronization.

---

14. Shared Printer ⋆⋆⋆⋆

Multiple threads send print jobs.

Allow only one thread to print at a time.

Part D – Inter-thread Communication

15. Producer-Consumer ⋆⋆⋆⋆

Create:

- Producer Thread
- Consumer Thread

Use:

- `wait()`
- `notify()`

---

16. Restaurant Order System ⋆⋆⋆⋆

Chef thread prepares food.

Customer thread waits until food is ready.

Use inter-thread communication.

---

17. Warehouse Simulation ⋆⋆⋆⋆

Producer adds products.

Consumer removes products.

Prevent overflow and underflow using:

- `wait()`
- `notifyAll()`

Part E – Executor Framework

18. Fixed Thread Pool ⋆⋆⋆⋆

Create an `ExecutorService` with a fixed thread pool.

Execute five independent tasks.

---

19. Task Scheduler ⋆⋆⋆⋆

Submit multiple tasks to an executor.

Display which thread executes each task.

Shutdown the executor properly.

Chapter Assignment ★★★★★

20. Online Food Delivery System

Build a console application.

Threads

- Customer Thread
- Restaurant Thread
- Chef Thread
- Delivery Partner Thread

Requirements

- Customer places an order.
- Restaurant receives the order.
- Chef prepares the food.
- Delivery partner delivers it.
- Customer receives the order.

Use

- Thread class
- Runnable interface
- `sleep()`
- `join()`
- Synchronization
- `wait()`
- `notify()`
- Executor Framework

Simulate multiple customers placing orders at the same time while ensuring correct synchronization.

Java 8+ Features – 20 Practice Problems

Part A – Lambda Expressions

1. First Lambda ⭑

Create a lambda expression to print:

```
Hello, Java 8!
```

---

2. Calculator using Lambda ⭑⭑

Create lambda expressions for:

- Addition
- Subtraction
- Multiplication
- Division

---

3. Sort Names ⭑⭑

Store student names in a list.

Sort them using a lambda expression.

---

4. Filter Even Numbers ⭑⭑⭑

Store integers in a list.

Display only even numbers using lambda expressions.

Part B – Functional Interfaces

5. Predicate Example ⭑⭑

Use `Predicate<Integer>` to check whether a number is even.

---

6. Function Example ⭑⭑⭑

Use `Function<String, Integer>` to return the length of a string.

---

7. Consumer Example ★★★

Store employee names in a list.

Print them using `Consumer`.

---

8. Supplier Example ★★★

Use `Supplier<String>` to generate random employee IDs.

---

9. Custom Functional Interface ★★★

Create your own functional interface.

Implement it using a lambda expression.

Part C – Stream API

10. Filter Students ★★★

Store student marks.

Display students scoring above 75 using Streams.

---

11. Sum of Numbers ★★★

Find the sum of all integers in a list using Stream API.

---

12. Remove Duplicates ★★★

Remove duplicate elements from a list using Streams.

---

13. Sort Products ★★★★

Create a `Product` class.

Sort products by price using Stream API.

---

14. Employee Statistics ★★★★

Store employees.

Display:

- Highest Salary
- Lowest Salary
- Average Salary
- Total Salary

using Streams.

Part D – Method References

15. Print Using Method Reference ★★★

Print all elements of a list using:

`System.out::println`

---

16. Constructor Reference ★★★★

Create objects using constructor references.

Part E – Default & Static Methods

17. Smart Device ★★★

Create an interface with:

- One abstract method
- One default method
- One static method

Implement it in a class.

---

18. Banking Interface ⋆⋆⋆⋆

Create an interface with:

- Default interest calculation
- Static bank information

Implement it in:

- SavingsAccount
- CurrentAccount

Part F – Optional Class

19. User Profile ⋆⋆⋆⋆

Create a `User` class.

Use `Optional` to safely handle:

- Name
- Email
- Phone Number

Avoid `NullPointerException`.

Chapter Assignment ⋆⋆⋆⋆⋆

20. Employee Management System (Java 8 Edition)

Build a console application.

Features

- Add Employee
- Remove Employee
- Search Employee
- Display Employees
- Sort Employees
- Filter Employees
- Salary Statistics

Requirements

Use:

- Lambda Expressions
- Functional Interfaces
- Stream API
- Method References
- Default Methods
- Static Methods
- Optional

Examples:

- Filter employees with salary > ₹50,000.
- Sort employees by salary and name.
- Find the highest-paid employee.
- Calculate average salary.
- Handle missing email IDs using `Optional`.
- Print employees using method references.

Inner Classes – 20 Practice Problems

Part A – Non-static Inner Class

1. Student Details ★

Create an outer class `Student`.

Create a non-static inner class `Address`.

Display complete student information.

---

2. Employee & Department ⋆⋆

Create:

- Outer Class → `Employee`
- Inner Class → `Department`

Display employee and department details.

---

3. Bank Account ⋆⋆

Create:

- `Bank`
- Inner class `Account`

Perform:

- Deposit
- Withdraw
- Balance Inquiry

---

4. Library Management ⋆⋆⋆

Create:

- `Library`
- Inner class `Book`

Display available books.

---

5. College Management ⋆⋆⋆

Create:

- `College`
- Inner class `Student`

Store multiple student objects.

Part B – Static Nested Class

6. University System ⋆⋆

Create:

- `University`
- Static nested class `Department`

Display department details without creating an outer class object.

---

7. Company Management ⋆⋆⋆

Create:

- `Company`
- Static nested class `Employee`

Display employee information.

---

8. Product Catalog ⋆⋆⋆

Create:

- `Store`
- Static nested class `Product`

Display product details.

---

9. Configuration Manager ⋆⋆⋆

Create:

- `Application`
- Static nested class `Configuration`

Store application settings.

---

10. Hospital System ⋆⋆⋆⋆

Create:

- `Hospital`
- Static nested class `Doctor`

Display doctor information.

Part C – Local Inner Class

11. Student Result ⋆⋆⋆

Inside a method,

Create a local inner class that calculates grades.

---

12. Salary Calculator ⋆⋆⋆

Inside a method,

Create a local inner class to calculate employee salary.

---

13. Shopping Bill ⋆⋆⋆

Inside a billing method,

Create a local inner class to calculate GST and final amount.

---

14. Electricity Bill ⋆⋆⋆⋆

Create a local inner class inside a billing method.

Calculate electricity charges.

---

15. Exam Report ⋆⋆⋆⋆

Create a local inner class inside a method to generate student reports.

Part D – Anonymous Inner Class

16. Greeting System ⋆⋆⋆

Create an interface.

Implement it using an anonymous inner class.

---

17. Calculator ⋆⋆⋆

Create an abstract class.

Implement one of its methods using an anonymous inner class.

---

18. Button Click Simulation ⋆⋆⋆⋆

Create an interface:

`ClickListener`

Implement it using an anonymous inner class.

Simulate a button click.

---

19. Shape Drawing ⋆⋆⋆⋆

Create an abstract class `Shape`.

Create anonymous objects for:

- Circle
- Rectangle

Override the drawing method.

Chapter Assignment ★★★★★

20. College Management Dashboard

Build a console application.

Requirements

Create an outer class:

`College`

Use:

Non-static Inner Class

- Student

Static Nested Class

- Department

Local Inner Class

Inside:

`generateReport()`

Generate student statistics.

Anonymous Inner Class

Create an interface:

`Notification`

Implement it anonymously to display:

- Admission Success
- Fee Payment Success
- Exam Registration Success

Features

- Add Student
- Add Department
- Display Student Details
- Generate Report
- Send Notification

Java Packages & Modifiers – 20 Practice Problems

Part A – Creating & Using Packages

1. First Package ⭐

Create a package named `student`.

Create a `Student` class inside it and display student details.

---

2. Multiple Packages ⭐⭐

Create two packages:

- `student`
- `teacher`

Create classes in both packages and use them in a main program.

---

3. Utility Package ⋆⋆

Create a package `utility`.

Add a `Calculator` class with methods:

- add()
- subtract()
- multiply()
- divide()

Import and use it.

---

4. Banking Package ⋆⋆⋆

Create packages:

- `bank.account`
- `bank.customer`

Create classes and demonstrate package organization.

---

5. College Management ⋆⋆⋆

Create packages:

- `college.student`
- `college.faculty`
- `college.department`

Create objects from all packages.

Part B – Import Statement

6. Import Single Class ⋆⋆

Import one class from another package and use it.

---

7. Import Entire Package ★★

Use `import packageName.*`.

Access multiple classes.

---

8. Static Import ★★★

Create a utility class with static methods.

Use static import to call methods without the class name.

---

9. Math Utility ★★★

Use static import for:

- `Math.sqrt()`
- `Math.pow()`
- `Math.max()`
- `Math.min()`

---

10. Multiple Package Integration ★★★

Create three packages.

Import classes from all of them into one application.

Part C – Access Modifiers

11. Public Access ★★

Create a public class.

Access it from another package.

Observe the behavior.

---

12. Private Access ⋆⋆⋆

Create private variables.

Access them using getters and setters.

---

13. Default Access ⋆⋆⋆

Create classes in the same package.

Demonstrate default access.

Then try accessing from another package and observe the result.

---

14. Protected Access ⋆⋆⋆

Create:

- Parent class
- Child class in another package

Access protected members through inheritance.

---

15. Compare Access Modifiers ⋆⋆⋆⋆

Create one program demonstrating:

- public
- protected
- default
- private

Compare accessibility from:

- Same class
- Same package
- Different package

- Subclass in different package

Part D – Mixed Package Problems

16. Employee Management ★★★

Create packages:

- `employee`
- `department`
- `payroll`

Use appropriate access modifiers.

Display employee details.

---

17. Library Management ★★★★

Organize classes into packages:

- `books`
- `members`
- `transactions`

Import and use them correctly.

---

18. E-Commerce Project Structure ★★★★

Create packages:

- `product`
- `customer`
- `order`
- `payment`

Create classes and integrate them.

---

19. Hospital Management ★★★★

Create packages:

- `doctor`
- `patient`
- `appointment`

Use:

- public
- protected
- default
- private

where appropriate.

Chapter Assignment ★★★★★

20. College ERP System

Build a package-based Java application.

Package Structure

```
college

├── student
│   └── Student.java

├── faculty
│   └── Faculty.java

├── department
│   └── Department.java

├── course
│   └── Course.java

├── utility
│   └── CollegeUtility.java

└── main
    └── Main.java
```

Requirements

- Organize classes into appropriate packages.
- Use `import` statements correctly.
- Demonstrate:
    - public access
    - protected access
    - default access
    - private access
- Use static import for utility methods.
- Create objects from different packages.
- Display complete college information.

Java Annotations – 20 Practice Problems

Part A – Built-in Annotations

1. First `@Override` ★

Create a parent class and child class.

Override a method using `@Override`.

---

2. Invalid `@Override` ★★

Intentionally use `@Override` on a method that does not override a parent method.

Observe the compiler error.

---

3. Shape Example ★★

Create:

- Shape
- Circle
- Rectangle

Override `draw()` using `@Override`.

---

4. Employee Management ★★

Create:

- Employee
- Manager
- Developer

Override `displayDetails()`.

---

5. Animal Sounds ★★★

Create:

- Animal
- Dog
- Cat
- Cow

Override `makeSound()`.

Part B – `@Deprecated`

6. Old Calculator ★★

Create an old method:

```
calculate()
```

Mark it as `@Deprecated`.

Create a new method:

```
calculateNew()
```

Use both methods.

---

7. Banking System ★★★

Create:

```
withdrawOld()
```

Mark it as deprecated.

Create:

`withdraw()`

Use the updated method.

---

8. Student Portal ⋆⋆⋆

Mark an old login method as deprecated.

Implement a new secure login method.

---

9. Library System ⋆⋆⋆

Deprecate an old search method.

Replace it with a new search implementation.

Part C – `@SuppressWarnings`

10. Raw ArrayList ⋆⋆⋆

Create a raw `ArrayList`.

Suppress the compiler warning using:

```
@SuppressWarnings("unchecked")
```

---

11. Deprecated Warning ⋆⋆⋆

Call a deprecated method.

Suppress the deprecation warning.

---

12. Multiple Warning Suppression ★★★

Suppress multiple compiler warnings in one program.

Part D – Custom Annotations

13. First Custom Annotation ★★★

Create your own annotation:

`@Author`

Store the author's name.

---

14. Version Annotation ★★★

Create:

`@Version`

Store version information for a class.

---

15. Developer Annotation ★★★★

Create:

`@Developer`

Store:

- Name
- Date
- Project

Apply it to multiple classes.

---

16. Student Information Annotation ★★★★

Create:

`@StudentInfo`

Store:

- Roll Number
- Name
- Branch

Apply it to a class.

---

17. Book Annotation ⋆⋆⋆⋆

Create:

`@BookInfo`

Store:

- Title
- Author
- Price

Apply it to a book class.

---

18. Employee Annotation ⋆⋆⋆⋆

Create:

`@EmployeeInfo`

Store:

- Employee ID
- Department
- Salary

Apply it to an employee class.

---

19. College Annotation System ★★★★

Create custom annotations for:

- Student
- Faculty
- Course

Annotate each corresponding class.

Chapter Assignment ★★★★★

20. University Management System

Build a small Java application.

Requirements

Create custom annotations:

- `@Author`
- `@Version`
- `@Department`
- `@CourseInfo`

Use built-in annotations:

- `@Override`
- `@Deprecated`
- `@SuppressWarnings`

Create classes:

- Student
- Faculty
- Course
- Department

Requirements:

- Override methods where appropriate.
- Mark old methods as deprecated.
- Suppress necessary warnings.

- Annotate classes with custom annotations containing metadata.

Java Date & Time API – 20 Practice Problems

Part A – LocalDate

1. Display Current Date ⭐

Display the current system date using `LocalDate`.

---

2. Date Information ⭐⭐

Display:

- Current Year
- Month
- Day
- Day of Week
- Day of Year

---

3. Birthday Calculator ⭐⭐

Accept your birth date.

Calculate your current age in years.

---

4. Days Until Birthday ⭐⭐⭐

Calculate how many days are left until your next birthday.

---

5. Leap Year Checker ⭐⭐⭐

Accept a year and determine whether it is a leap year using `LocalDate`.

Part B – LocalTime & LocalDateTime

6. Display Current Time ⋆⋆

Display:

- Current Hour
- Minute
- Second

using `LocalTime`.

---

7. Digital Clock ⋆⋆⋆

Display the current date and time using `LocalDateTime`.

---

8. Office Working Hours ⋆⋆⋆

Accept office start and end times.

Display the total working hours.

---

9. Event Reminder ⋆⋆⋆

Accept an event date and time.

Display whether the event is:

- Upcoming
- Ongoing
- Completed

---

10. Student Attendance Time ⋆⋆⋆

Store login and logout times.

Calculate total attendance duration.

Part C – Period & Duration

11. Age Calculator ⋆⋆

Use `Period` to calculate:

- Years
- Months
- Days

between birth date and today.

---

12. Subscription Validity ⋆⋆⋆

Accept:

- Subscription Start Date
- Subscription End Date

Display the remaining validity period.

---

13. Movie Duration ⋆⋆⋆

Accept movie start and end times.

Calculate the total duration using `Duration`.

---

14. Online Exam Timer ⋆⋆⋆⋆

Accept:

- Exam Start Time

- Exam End Time

Display:

- Hours
- Minutes
- Seconds

remaining.

---

15. Workout Tracker ⋆⋆⋆⋆

Store exercise start and end times.

Calculate the workout duration.

Part D – DateTimeFormatter

16. Format Current Date ⋆⋆⋆

Display today's date in different formats:

Examples:

- `dd/MM/yyyy`
- `MM-dd-yyyy`
- `dd-MMM-yyyy`
- `EEEE, dd MMMM yyyy`

---

17. Parse User Date ⋆⋆⋆

Accept a date in:

```
25-12-2026
```

Convert it into a `LocalDate` object.

---

18. Meeting Scheduler ★★★★

Accept date and time from the user.

Format and display them in multiple readable styles.

---

19. Employee Joining Report ★★★★

Store employee joining dates.

Display:

- Joining Date
- Experience (Years/Months)
- Formatted Date

Chapter Assignment ★★★★★

20. Appointment Management System

Build a console application.

Features

- Add Appointment
- View Appointment
- Calculate Remaining Days
- Check Expired Appointment
- Format Appointment Date
- Exit

Requirements

Use:

- `LocalDate`
- `LocalTime`
- `LocalDateTime`
- `Period`
- `Duration`
- `DateTimeFormatter`

Display:

- Appointment Date
- Appointment Time
- Days Remaining
- Appointment Duration (if applicable)
- Formatted Output

Java Utility Classes – 20 Practice Problems

Part A – Math Class

1. Math Operations ⋆

Accept two numbers and display:

- Maximum
- Minimum
- Absolute Value
- Square Root
- Power

using the `Math` class.

---

2. Circle Calculator ⋆⋆

Accept the radius and calculate:

- Area
- Circumference

Use `Math.PI`.

---

3. Scientific Calculator ⋆⋆⋆

Create a calculator supporting:

- Square
- Cube
- Power
- Square Root

- Ceiling
- Floor
- Round

using the `Math` class.

Part B – Arrays Utility Class

4. Sort an Array ⋆⋆

Accept an integer array.

Sort it using `Arrays.sort()`.

---

5. Compare Arrays ⋆⋆⋆

Accept two arrays.

Check whether they are equal using `Arrays.equals()`.

---

6. Copy Arrays ⋆⋆⋆

Create copies of an array using:

- `Arrays.copyOf()`
- `Arrays.copyOfRange()`

---

7. Binary Search ⋆⋆⋆

Sort an array.

Search for an element using `Arrays.binarySearch()`.

Part C – Collections Utility Class

8. Student Marks List ★★★

Store student marks in an `ArrayList`.

Use `Collections` methods to:

- Sort
- Reverse
- Shuffle

---

9. Highest & Lowest Marks ★★★

Find:

- Maximum
- Minimum

using `Collections.max()` and `Collections.min()`.

---

10. Frequency Counter ★★★★

Store duplicate values in a list.

Find the frequency of a given element using `Collections.frequency()`.

---

11. Rotate List ★★★★

Rotate a list using `Collections.rotate()`.

Display the list before and after rotation.

Part D – Objects Utility Class

12. Null Check ★★

Create an object reference.

Check whether it is null using `Objects.isNull()` and `Objects.nonNull()`.

---

13. Safe Comparison ★★★

Compare two objects safely using `Objects.equals()`.

---

14. Student Equality ★★★

Create a `Student` class.

Compare two student objects using `Objects.equals()`.

---

15. Default Value ★★★★

Use `Objects.requireNonNullElse()` to provide default values when an object is null.

Part E – Scanner, Random & StringTokenizer

16. Student Information System ★★

Use `Scanner` to accept:

- Name
- Age
- Branch
- CGPA

Display the information.

---

17. Dice Rolling Simulator ★★★

Use `Random` to simulate rolling a six-sided dice 100 times.

Display the frequency of each face.

---

18. Password Generator ★★★★

Generate random passwords containing:

- Uppercase letters
- Lowercase letters
- Digits

Use the `Random` class.

---

19. String Tokenizer ★★★★

Accept the sentence:

```
Java is a powerful programming language
```

Use `StringTokenizer` to:

- Count tokens
- Print each token
- Display the longest word

Chapter Assignment ★★★★★

20. Student Utility Toolkit

Build a console application.

Features

- Student Information Input
- Marks Analysis
- Random Roll Number Generator
- Subject Name Tokenizer
- Marks Sorting
- Highest & Lowest Marks
- Binary Search for Marks

- Null-safe Student Comparison

Requirements

Use:

- `Math`
- `Arrays`
- `Collections`
- `Objects`
- `Scanner`
- `Random`
- `StringTokenizer`

Functionalities

- Accept student details using `Scanner`.
- Generate a random student ID.
- Sort and search marks.
- Find highest and lowest marks.
- Tokenize subject names entered as a single string.
- Compare student objects safely.
- Handle null values using `Objects`.

Enum in Java – 20 Practice Problems

Part A – Enum Basics

1. Days of the Week ⭑

Create an enum `Day` containing:

- MONDAY
- TUESDAY
- WEDNESDAY
- THURSDAY
- FRIDAY
- SATURDAY
- SUNDAY

Display all values.

---

2. Traffic Signal ⭑⭑

Create an enum:

`TrafficSignal`

Values:

- RED
- YELLOW
- GREEN

Use `switch` to display the action for each signal.

---

3. Months of the Year ⋆⋆

Create an enum `Month`.

Accept a month and display:

- Month Name
- Month Number

---

4. Seasons ⋆⋆

Create an enum:

- SUMMER
- WINTER
- SPRING
- AUTUMN

Display a suitable message for each season.

---

5. Student Grade ⋆⋆⋆

Create an enum:

- A
- B
- C
- D

- F

Display remarks for each grade.

Part B – Enum Methods

6. Enum Explorer ⋆⋆⋆

Using an enum, demonstrate:

- `values()`
- `valueOf()`
- `ordinal()`
- `name()`

---

7. Employee Department ⋆⋆⋆

Create an enum:

- HR
- SALES
- FINANCE
- IT

Display each department with its ordinal value.

---

8. User Role ⋆⋆⋆

Create an enum:

- ADMIN
- MANAGER
- EMPLOYEE
- GUEST

Display permissions using `switch`.

---

9. Online Order Status ⋆⋆⋆

Create an enum:

- ORDERED
- SHIPPED
- OUT_FOR_DELIVERY
- DELIVERED
- CANCELLED

Display an appropriate message for each status.

---

10. Payment Status ⋆⋆⋆

Create an enum:

- PENDING
- SUCCESS
- FAILED
- REFUNDED

Display the corresponding transaction message.

Part C – Enum with Constructor & Methods

11. Planet Information ⋆⋆⋆

Create an enum `Planet`.

Store:

- Planet Name
- Gravity

Display both values.

---

12. Vehicle Type ⋆⋆⋆

Create an enum:

- BIKE
- CAR
- BUS

Store:

- Number of Wheels

Display the details.

---

13. Product Category ⋆⋆⋆⋆

Create an enum:

- ELECTRONICS
- GROCERY
- CLOTHING

Store:

- GST Percentage

Display category and tax.

---

14. Cricket Format ⋆⋆⋆⋆

Create an enum:

- TEST
- ODI
- T20

Store:

- Number of Overs

Display the format and overs.

---

15. Course Details ⋆⋆⋆⋆

Create an enum:

- JAVA
- PYTHON
- C++
- WEB_DEVELOPMENT

Store:

- Course Duration
- Fee

Display all details.

Part D – Real-world Enum Applications

16. ATM Transaction ⋆⋆⋆⋆

Create an enum:

- DEPOSIT
- WITHDRAW
- BALANCE_ENQUIRY
- EXIT

Implement a menu-driven ATM using the enum.

---

17. Employee Attendance ⋆⋆⋆⋆

Create an enum:

- PRESENT
- ABSENT
- LEAVE
- HALF_DAY

Display salary deduction rules based on attendance.

---

18. Online Examination ⋆⋆⋆⋆

Create an enum:

- NOT_STARTED
- IN_PROGRESS
- SUBMITTED
- EVALUATED

Display appropriate messages for each exam status.

---

19. Library Management ⋆⋆⋆⋆

Create an enum:

- AVAILABLE
- ISSUED
- RESERVED
- LOST

Display the availability status of books.

Chapter Assignment ⋆⋆⋆⋆⋆

20. E-Commerce Order Management System

Build a console application.

Features

- Place Order
- Update Order Status
- View Order
- Cancel Order
- Exit

Create Enums

OrderStatus

- ORDERED
- PACKED
- SHIPPED

- OUT_FOR_DELIVERY
- DELIVERED
- CANCELLED

PaymentStatus

- PENDING
- SUCCESS
- FAILED
- REFUNDED

UserRole

- CUSTOMER
- SELLER
- ADMIN

Requirements

- Use enums in menu handling.
- Create enums with constructors and methods.
- Store additional information inside enums.
- Display enum details using:
  - `values()`
  - `ordinal()`
  - `name()`
- Use `switch` with enums.
- Demonstrate practical enum usage in a real-world application.

Static & Instance Initialization – 20 Practice Problems

Part A – Static Initializer Blocks

1. First Static Block ⭑

Create a class containing:

- One static block
- `main()` method

Observe which executes first.

---

2. Multiple Static Blocks ⋆⋆

Create a class with three static blocks.

Display their execution order.

---

3. Static Block with Static Variable ⋆⋆

Initialize a static variable inside a static block.

Display the value in `main()`.

---

4. Static Block & Static Method ⋆⋆⋆

Call a static method from a static block.

Observe the execution sequence.

---

5. Student Counter ⋆⋆⋆

Create a `Student` class.

Initialize the college name inside a static block.

Display it using multiple objects.

Part B – Instance Initializer Blocks

6. First Instance Block ⋆⋆

Create:

- One instance initializer block
- Constructor

Observe the execution order.

---

7. Multiple Instance Blocks ⋆⋆⋆

Create two instance initializer blocks.

Create multiple objects and observe how many times they execute.

---

8. Instance Block & Constructor ⋆⋆⋆

Initialize instance variables inside the instance initializer block.

Display them using the constructor.

---

9. Employee Record ⋆⋆⋆

Initialize employee details using an instance initializer block.

Display them through objects.

---

10. Product Inventory ⋆⋆⋆

Assign default product values using an instance initializer block.

Override some values through constructors.

Part C – Static vs Instance Initialization

11. Execution Order ⋆⋆⋆

Create:

- Static block
- Instance block

- Constructor
- `main()`

Print messages from each and observe the exact execution order.

---

12. Multiple Objects ⋆⋆⋆

Create three objects.

Observe:

- Static block execution count
- Instance block execution count
- Constructor execution count

---

13. Banking System ⋆⋆⋆⋆

Initialize:

- Bank Name → Static Block
- Account Holder Details → Instance Block

Display both.

---

14. Hospital Management ⋆⋆⋆⋆

Create:

- Static block for hospital information.
- Instance block for patient information.

Display all details.

---

15. Vehicle Registration ⋆⋆⋆⋆

Use:

- Static block → Registration Office
- Instance block → Vehicle Details

Create multiple vehicle objects.

Part D – Mixed Practice

16. Library Management ⋆⋆⋆

Initialize:

- Library Name → Static Block
- Book Details → Instance Block

---

17. College Management ⋆⋆⋆⋆

Initialize:

- College Name → Static Block
- Student Details → Instance Block

Create multiple student objects.

---

18. Company Payroll ⋆⋆⋆⋆

Initialize:

- Company Name → Static Block
- Employee Details → Instance Block

Display salary details.

---

19. University System ⋆⋆⋆⋆

Create:

- Static Block
- Instance Block
- Static Variable
- Instance Variable
- Constructor

Print messages to clearly demonstrate the complete execution flow.

Chapter Assignment ★★★★★

20. School Management System

Build a console application.

Requirements

Create a `School` class.

Use:

Static Initializer Block

Initialize:

- School Name
- School Code
- Principal Name

Instance Initializer Block

Initialize:

- Student Roll Number
- Name
- Class
- Section

Constructor

Accept:

- Marks
- Grade

Display

- School Information
- Student Information
- Marks
- Grade

Create multiple student objects and clearly observe:

- Static block executes only once.
- Instance block executes for every object.
- Constructor executes for every object after the instance block.

Java Applets – 20 Practice Problems

Part A – Applet Basics

1. My First Applet ⭐

Create a simple applet that displays:

```
Welcome to Java Applet
```

---

2. Personal Information ⭐

Display:

- Name
- Roll Number
- Branch
- College

inside an applet.

---

3. Draw a Line ⭐⭐

Draw a straight line using the `Graphics` class.

---

4. Draw Basic Shapes ⋆⋆

Draw:

- Rectangle
- Oval
- Circle
- Square

using graphics methods.

---

5. Display Colored Text ⋆⋆

Display text in different colors using `Graphics`.

Part B – Lifecycle Methods

6. init() Demonstration ⋆⋆

Override `init()`.

Display a message showing it executes first.

---

7. start() Demonstration ⋆⋆

Override `start()`.

Print a message when the applet starts.

---

8. stop() Demonstration ⋆⋆⋆

Override `stop()`.

Display a message when the applet stops.

---

9. destroy() Demonstration ⋆⋆⋆

Override `destroy()`.

Display a cleanup message.

---

10. Complete Lifecycle ⋆⋆⋆

Override:

- init()
- start()
- paint()
- stop()
- destroy()

Display the execution order.

Part C – Graphics

11. Draw National Flag ⋆⋆⋆

Draw a simple flag using graphics methods.

---

12. Traffic Signal ⋆⋆⋆

Draw a traffic signal using circles.

---

13. Smiley Face ⋆⋆⋆

Draw a smiley face using:

- Circle
- Eyes
- Smile

---

14. House Drawing ⋆⋆⋆⋆

Draw a house using basic graphics primitives.

---

15. Olympic Rings ⋆⋆⋆⋆

Draw the Olympic rings using colored circles.

Part D – User Interaction

16. Display Current Date ⋆⋆⋆

Display today's date inside an applet.

---

17. Addition Applet ⋆⋆⋆

Accept two numbers.

Display their sum.

---

18. Student Grade Applet ⋆⋆⋆⋆

Accept marks.

Calculate:

- Total
- Percentage
- Grade

Display the result.

---

19. Mini Calculator ⋆⋆⋆⋆

Create a calculator supporting:

- Addition
- Subtraction
- Multiplication
- Division

inside an applet.

Chapter Assignment ⋆⋆⋆⋆⋆

20. Student Information Applet

Build an applet.

Features

- Display student details.
- Draw a border and heading.
- Show the current date.
- Calculate total and percentage.
- Draw simple graphics (line, rectangle, circle).
- Override all lifecycle methods:
  - `init()`
  - `start()`
  - `paint()`
  - `stop()`
  - `destroy()`

Command Line Arguments – 20 Practice Problems

Part A – Basics

1. Print All Arguments ⋆

Write a program that prints all command-line arguments passed to `main(String[] args)`.

---

2. Count Arguments ⭑

Display the total number of command-line arguments.

---

3. Personal Information ⭑⭑

Pass the following as command-line arguments:

- Name
- Age
- Branch
- College

Display them in a formatted manner.

---

4. Greeting Program ⭑⭑

Accept a user's name as a command-line argument and display:

```
Welcome, <Name>!
```

---

5. Simple Calculator ⭑⭑

Accept:

- Number 1
- Operator
- Number 2

from the command line and perform the operation.

Part B – Numeric Arguments

6. Sum of Numbers ⋆⋆

Accept multiple integers through command-line arguments and calculate their sum.

---

7. Average Calculator ⋆⋆⋆

Calculate the average of all numeric command-line arguments.

---

8. Largest Number ⋆⋆⋆

Find the largest number among the command-line arguments.

---

9. Even & Odd Counter ⋆⋆⋆

Count how many command-line arguments are even and how many are odd.

---

10. Factorial Calculator ⋆⋆⋆

Accept one integer from the command line and calculate its factorial.

Part C – String Arguments

11. Reverse a String ⋆⋆

Accept a string as a command-line argument and print it in reverse.

---

12. Palindrome Checker ⋆⋆⋆

Check whether the string passed through the command line is a palindrome.

---

13. Word Counter ⋆⋆⋆

Pass a sentence (properly quoted in the terminal) and display the number of words.

---

14. Vowel Counter ⋆⋆⋆

Count the number of vowels in the string passed through command-line arguments.

---

15. Username & Password Validation ⋆⋆⋆⋆

Accept:

- Username
- Password

through command-line arguments and validate them.

Part D – Mixed Applications

16. Student Result ⋆⋆⋆

Accept:

- Name
- Roll Number
- Marks of 5 Subjects

through command-line arguments.

Calculate:

- Total
- Percentage
- Grade

---

17. Currency Converter ⋆⋆⋆⋆

Accept:

- Amount
- Conversion Type (USD, INR, EUR, etc.)

through command-line arguments and display the converted amount using predefined rates.

---

18. Employee Salary Calculator ★★★★

Accept:

- Employee Name
- Basic Salary
- HRA
- DA

through command-line arguments.

Calculate and display the net salary.

---

19. Mini Login System ★★★★

Accept:

- Username
- Password
- Role

through command-line arguments.

Display an appropriate welcome message if the credentials are valid.

Chapter Assignment ★★★★★

20. Student Management System (Command-Line Version)

Build a Java application that receives all input through command-line arguments.

Input

- Student Name
- Roll Number
- Branch
- Semester
- Marks in 5 Subjects

Features

- Display student details
- Calculate total marks
- Calculate percentage
- Assign grade
- Display pass/fail status
- Validate the number of command-line arguments
- Display meaningful error messages if arguments are missing or invalid

Concepts Covered

- `main(String[] args)`
- Reading command-line arguments
- Converting `String` arguments to numeric types
- Input validation
- Basic calculations
- Decision-making (`if`/`switch`)
- Exception handling for invalid input

Remaining Core Java Topics – 30 Practice Problems

Methods

1. Simple Calculator ⋆

Create methods for:

- Addition
- Subtraction
- Multiplication
- Division

---

2. Maximum of Three Numbers ⋆

Create a method that returns the largest of three numbers.

---

3. Prime Number Method ⋆⋆

Create a method `isPrime(int n)` that returns `true` if the number is prime.

---

4. Armstrong Number Method ⋆⋆

Create a method that checks whether a number is an Armstrong number.

---

5. Method Overloading ⋆⋆

Overload an `area()` method for:

- Circle
- Rectangle
- Triangle

---

6. Varargs Calculator ⋆⋆⋆

Create a method that accepts any number of integers using varargs and returns their sum.

---

7. Student Result Methods ⋆⋆⋆

Separate the following into methods:

- Input
- Total
- Percentage
- Grade
- Display

---

8. Call by Value Demonstration ⋆⋆⋆

Write a program proving Java is pass-by-value.

Recursion

9. Factorial ⋆⋆

Find factorial using recursion.

---

10. Fibonacci ⋆⋆

Print the first N Fibonacci numbers recursively.

---

11. Sum of Digits ⋆⋆⋆

Find the sum of digits recursively.

---

12. Reverse Number ⋆⋆⋆

Reverse a number using recursion.

---

13. String Reverse ⋆⋆⋆

Reverse a string recursively.

---

14. Palindrome String ⋆⋆⋆

Check whether a string is a palindrome using recursion.

---

15. Power Function ★★★★

Calculate aba^bab recursively.

Object Class

16. Override `toString()` ★★

Create a `Student` class and override `toString()`.

---

17. Override `equals()` ★★★

Compare two student objects based on roll number.

---

18. Override `hashCode()` ★★★

Generate hash codes for objects and observe their behavior.

---

19. `getClass()` Explorer ★★★

Display the runtime class information of different objects.

---

20. Clone Object ★★★★

Create a clone of a `Student` object and compare the original and cloned objects.

Searching & Sorting

21. Linear Search ★★

Search for an element in an array.

---

22. Binary Search ★★★

Perform binary search after sorting the array.

---

23. Bubble Sort ★★★

Sort an array in ascending order using Bubble Sort.

---

24. Selection Sort ★★★

Sort an array using Selection Sort.

---

25. Insertion Sort ★★★

Sort an array using Insertion Sort.

Memory & Garbage Collection

26. Object Creation Tracker ★★★

Create multiple objects and print messages from constructors to observe object creation.

---

27. Garbage Collection Demo ★★★

Create unused objects and invoke `System.gc()`.

Observe the behavior (without assuming it will always run immediately).

---

28. String Pool Demonstration ★★★★

Compare different string creation techniques using:

- ==

- `equals()`

Observe String Pool behavior.

Debugging & Output Prediction

29. Debug the Program ★★★★

Given a Java program containing:

- Compilation errors
- Logical errors
- Runtime errors

Identify and fix all issues.

Final Capstone Assignment ★★★★★

30. Student Management System (Core Java Edition)

Build a complete console application using everything learned in Core Java.

Features

- Add Student
- Update Student
- Delete Student
- Search Student
- Display All Students
- Sort Students by Marks
- Search by Roll Number
- Calculate Grades
- Save Student Data to File
- Read Student Data from File
- Handle Invalid Input using Exceptions

Mandatory Concepts

- Methods
- Recursion (at least one feature)
- Arrays
- OOP

- Constructors
- Inheritance
- Polymorphism
- Encapsulation
- Abstract Classes / Interfaces
- Strings
- Exception Handling
- Wrapper Classes
- Collections (`ArrayList`, `HashMap`)
- File Handling
- Generics
- Enums
- Date & Time API
- Utility Classes
- Static Members
- Packages
- Object Class (`toString()`, `equals()`, `hashCode()`)
- Searching
- Sorting